

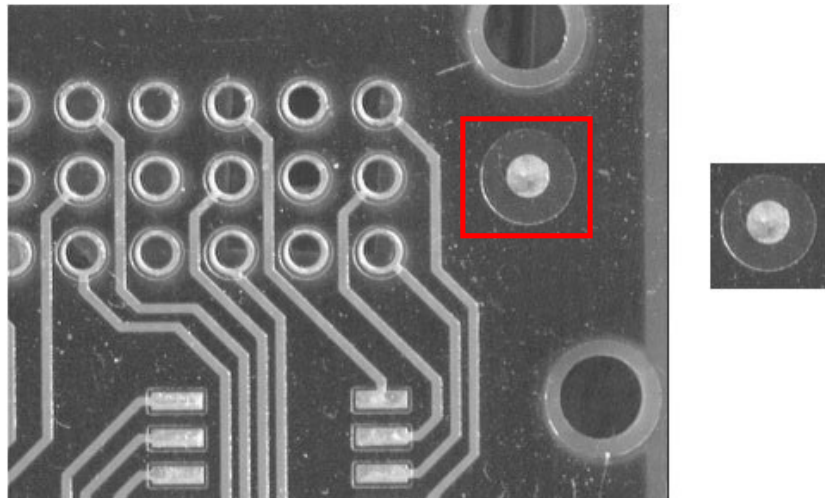
Instance-level Object Detection

Prof. Giuseppe Lisanti

giuseppe.lisanti@unibo.it

Instance-level Object Detection

- The *Instance-level Object Detection* problem occurs in countless applications and can be formulated as follows.
 - Given a reference image (aka model image) of a *specific object*, determine whether the object is present or not in the image under analysis (aka target image) and, in case of detection, estimate the pose of the object.



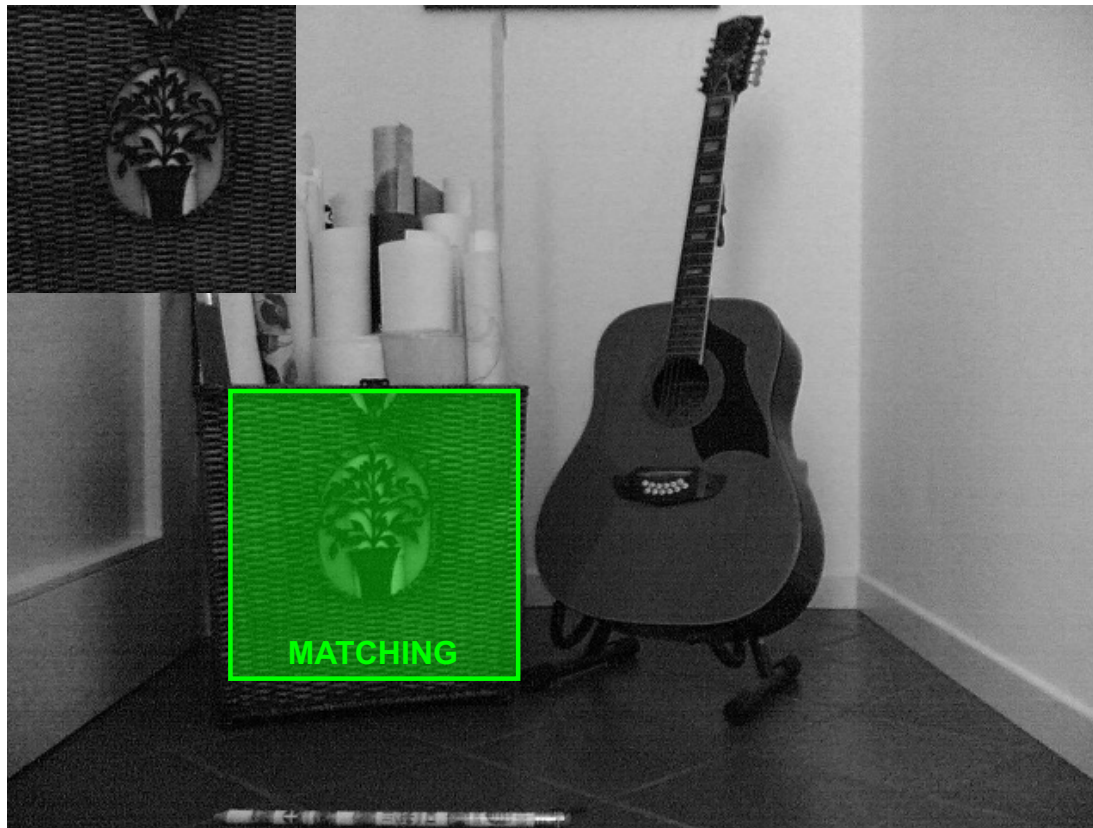
- Depending on the application, the pose may often be given by a translation, a roto-translation or a similarity (roto-translation plus scale)

Instance-level Object Detection

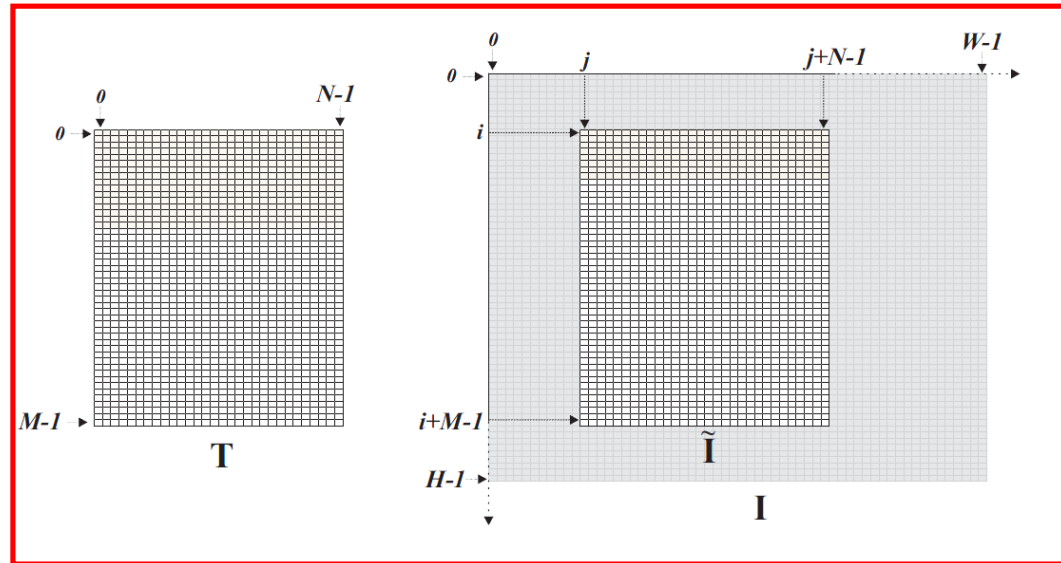
- The problem has a number of diverse facets: the sought object may appear only once or multiple times in the target image, we may be interested in detecting a number of diverse objects, each of which, in turn, may appear once or multiple times.
 - For the sake of simplicity, and without much loss of generality, we will refer mainly to the basic setting: *detecting a single object that may appear once in the target image*. Generalization to the other variants turns out more often than not straightforward.
- Typical nuisances to be dealt with are *intensity changes, occlusions and clutter*.
- Computational efficiency is a major requirement in most practical applications.
- This problem is characterized by a *limited variability* as the assumption deals with the appearance of the object being captured by a single model image (i.e. roughly planar objects or no view-point changes) and the pose is typically either a 2D translation or a 2D roto-translation or a similarity.
- Given the limited variability, the problem can be addressed successfully by classical computer vision techniques, the major applications dealing mainly with the space of industrial vision.
- Instead, Category-level Object Detection aims at detecting certain kind of object(s) (e.g. cars, pedestrians..) regardless of their appearance and pose. Due to the high-variability, this problem is addressed by machine/deep learning techniques.

Template Matching

- The model image is slid across the target image to be compared at each position to an equally sized window by means of a suitable (dis)similarity function



(Dis)Similarity Functions



- $\tilde{I}(i, j)$, the window at position (i, j) of the target image has the same size as **T**

- Compute pixel-wise intensities differences:

$$SSD(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} \left(I(i+m, j+n) - T(m, n) \right)^2$$

(Dis)Similarity Functions

- Sum of Absolute Differences: $SAD(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} |I(i+m, j+n) - T(m, n)|$

Since difference of intensities are used but with 2 different images (I and T), we can't assume they have the same additive (constant gain) nor multiplicative (linear gain) bias and so they are invariant to both.

- Are SSD and SAD invariant to intensity changes? **NO, why? when we should use them?**

- Normalised Cross-Correlation:
$$NCC(i, j) = \frac{\sum_{m=0}^{M-1} \sum_{n=0}^{N-1} I(i+m, j+n) \cdot T(m, n)}{\sqrt{\sum_{m=0}^{M-1} \sum_{n=0}^{N-1} I(i+m, j+n)^2} \cdot \sqrt{\sum_{m=0}^{M-1} \sum_{n=0}^{N-1} T(m, n)^2}}$$

Similarity!

In this way it is not important the intensity of the pixels

- Represents the cosine of the angle between vectors $I(i, j)$ e T

$$NCC(i, j) = \frac{\tilde{\mathbf{I}}(i, j) \cdot \mathbf{T}}{\|\tilde{\mathbf{I}}(i, j)\| \cdot \|\mathbf{T}\|} = \frac{\cancel{\|\tilde{\mathbf{I}}(i, j)\|} \cdot \cancel{\|\mathbf{T}\|} \cdot \cos \theta}{\cancel{\|\tilde{\mathbf{I}}(i, j)\|} \cdot \cancel{\|\mathbf{T}\|}} = \cos \theta$$

- Invariant to linear intensity changes $\tilde{\mathbf{I}}(i, j) = \alpha \cdot \mathbf{T}$

!! Still not invariant to constant gain !!

Using centered version of the image
 $(I' - \mu(I'))$ we solve the additive gain problem
 since if $I' = I + b$, then $\mu(I') = \mu(I) + b$.

So
 $I' - \mu(I') = I - \mu(I)$
 which is the centered version without
 additive gain, so it is invariant to it.

(Dis)Similarity Functions

- Zero-Mean Normalised Cross-Correlation, Correlation Coefficient

$$\mu(\tilde{I}) = \frac{1}{MN} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} I(i+m, j+n)$$

$$\mu(T) = \frac{1}{MN} \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} T(m, n)$$

- The NCC is computed after subtraction of the means:

Mean of the subimage

$$I(i+m, j+n) \rightarrow (I(i+m, j+n) - \mu(\tilde{I}))$$

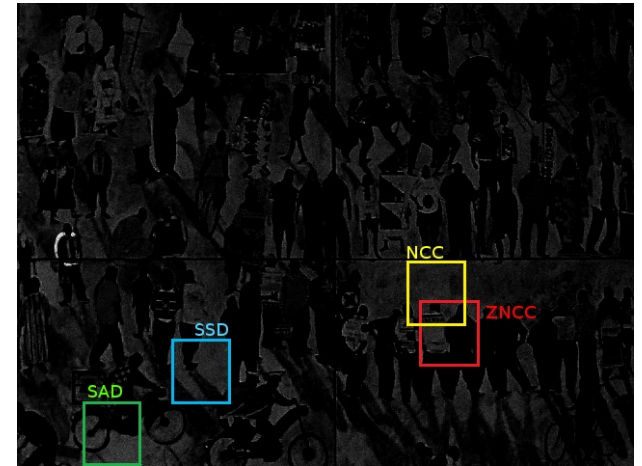
$$T(m, n) \rightarrow (T(m, n) - \mu(T))$$



$$ZNCC(i, j) = \frac{\sum_{m=0}^{M-1} \sum_{n=0}^{N-1} (I(i+m, j+n) - \mu(\tilde{I})) \cdot (T(m, n) - \mu(T))}{\sqrt{\sum_{m=0}^{M-1} \sum_{n=0}^{N-1} (I(i+m, j+n) - \mu(\tilde{I}))^2} \cdot \sqrt{\sum_{m=0}^{M-1} \sum_{n=0}^{N-1} (T(m, n) - \mu(T))^2}}$$

- Invariant to affine intensity changes $\tilde{I}(i, j) = \alpha \cdot T + \beta$

Comparison under significant intensity changes



ZNCC turns out to be a similarity function very robust to intensity changes!

Shape-based Matching

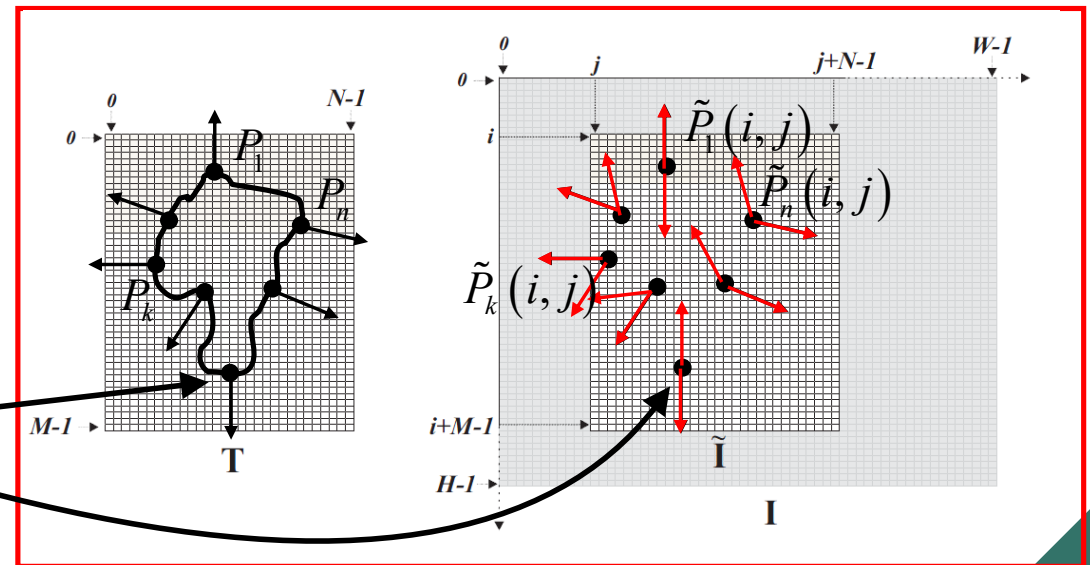
It is still a template matching approach but more focused on the shapes of the template rather than the simple illumination of the template

- Edge-based template matching approach
 - First, a set of control points, P_k , is extracted from the model image by an Edge Detector and the gradient direction at each P_k is stored
 - Template composed by offsets and gradient directions
 - Then, at each position (i,j) of the target image, the **recorded** gradient directions associated with control points are compared to those at their corresponding image points, $P_k(i,j)$

- The more the red arrows are aligned to the black arrows the more similar the sub-image is to the template

- We do not perform edge detection on the target image, just on the template

Given the P_k points of the template, we simply consider their respective points in the considered portion of the image



Similarity Function

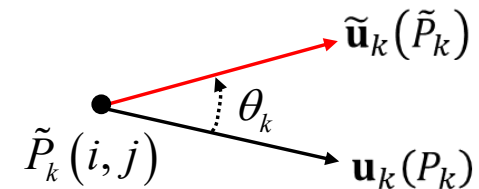
Normalized => unit vector

$$\begin{array}{lcl}
 \boxed{\mathbf{G}_k(P_k)} = \begin{bmatrix} I_x(P_k) \\ I_y(P_k) \end{bmatrix}, & \boxed{\mathbf{u}_k(P_k)} = \frac{1}{\|\mathbf{G}_k(P_k)\|} \begin{bmatrix} I_x(P_k) \\ I_y(P_k) \end{bmatrix}, & k = 1..n \quad \text{Template} \\
 \boxed{\tilde{\mathbf{G}}_k(\tilde{P}_k)} = \begin{bmatrix} I_x(\tilde{P}_k) \\ I_y(\tilde{P}_k) \end{bmatrix}, & \boxed{\tilde{\mathbf{u}}_k(\tilde{P}_k)} = \frac{1}{\|\tilde{\mathbf{G}}_k(\tilde{P}_k)\|} \begin{bmatrix} I_x(\tilde{P}_k) \\ I_y(\tilde{P}_k) \end{bmatrix}, & k = 1..n \quad \text{Target} \\
 \text{Intensities} & \text{Directions} &
 \end{array}$$

$$\|\mathbf{u}\| \cdot \|\mathbf{v}\| \cdot \cos = \cos$$

$$S(i, j) = \frac{1}{n} \sum_{k=1}^n \mathbf{u}_k(P_k) \cdot \tilde{\mathbf{u}}_k(\tilde{P}_k) = \frac{1}{n} \sum_{k=1}^n \cos \theta_k$$

- The similarity function spans the interval [-1; 1]
 - It takes its maximum value when all the gradients at the control points in the current window of the target image are perfectly aligned to those at the control points of the model image
 - If they are perfectly aligned the angle is zero, the cosine is 1, the sum is n, which divided by n is 1, and vice versa
- Choosing a detection threshold, $S_{\min}$, can be thought of as specifying the fraction of model points which must be seen in the image to trigger a detection



More robust similarity functions

- Certain application settings call for invariance to *global inversion of contrast polarity* along object's contours, as the object may appear either darker or brighter than the background in the target image
- This kind of invariance can be achieved by a slight modification to the similarity function defined previously (global)

↓
Useful when the whole image is inverted.
From B/W to W/B. (eg bones in X-rays)

$$S(i, j) = \frac{1}{n} \left| \sum_{k=1}^n \mathbf{u}_k(P_k) \cdot \tilde{\mathbf{u}}_k(\tilde{P}_k) \right| = \frac{1}{n} \left| \sum_{k=1}^n \cos \theta_k \right|$$

The result is positive but inside there can be subtractions

- The following function is even more robust due to the ability to withstand local contrast polarity inversions (local)

↓
Useful when parts of the image got inverted maybe because of reflections or shadows

$$S(i, j) = \frac{1}{n} \sum_{k=1}^n |\mathbf{u}_k(P_k) \cdot \tilde{\mathbf{u}}_k(\tilde{P}_k)| = \frac{1}{n} \sum_{k=1}^n |\cos \theta_k|$$

The result is positive because there are only sums

The Hough Transform

Templates with regular shapes

- The *Hough Transform (HT)* enables to detect objects having a known shape that *can be expressed by an equation* (e.g. lines, circles, ellipses..) based on projection of the input data into a suitable space referred to as *parameter* or *Hough space* (different from the image space)
- The HT turns a global detection problem into a local one (i.e., look for feature points into the parameter space, instead of looking for the whole shape in the image space)
- The HT is usually applied after an edge detection process (i.e., the actual input data consist of the edge pixels extracted from the original image)
- The HT is robust to noise and allows for detecting the sought shape even though it is partially occluded into the image (up to a certain user-selectable degree of occlusion)
- The HT was invented to detect lines and later extended to other analytical shapes (circle, ellipses) as well as to *arbitrary shapes => Generalized Hough Transform (GHT)*
- The GHT principle is widely deployed also within object detection pipelines relying on local invariant features such as, e.g., SIFT

Usually we represent the space ($y-mx-c=0$) with fixed (m,c) , now we fix (x,y) :

- Before we were looking for all the space of all points (x,y) passing for a specific line (m,c)
- Now we look for all the lines (m,c) passing through a specific point (x,y)

Basic Principle

(simplest example with the line equation)

- HT formulation for lines, what is fixed and what is changing in this equation? $y - mx - c = 0$

- In the usual image space interpretation of the line equation the parameters $(\hat{m}, \hat{c})$ are fixed

$$y - \hat{m}x - \hat{c} = 0$$

so that the equation represents the mapping from point $(\hat{m}, \hat{c})$ of the *parameter space to the image points belonging to the line*

- However, we may instead fix $(\hat{x}, \hat{y})$

$$\hat{y} - m\hat{x} - c = 0$$

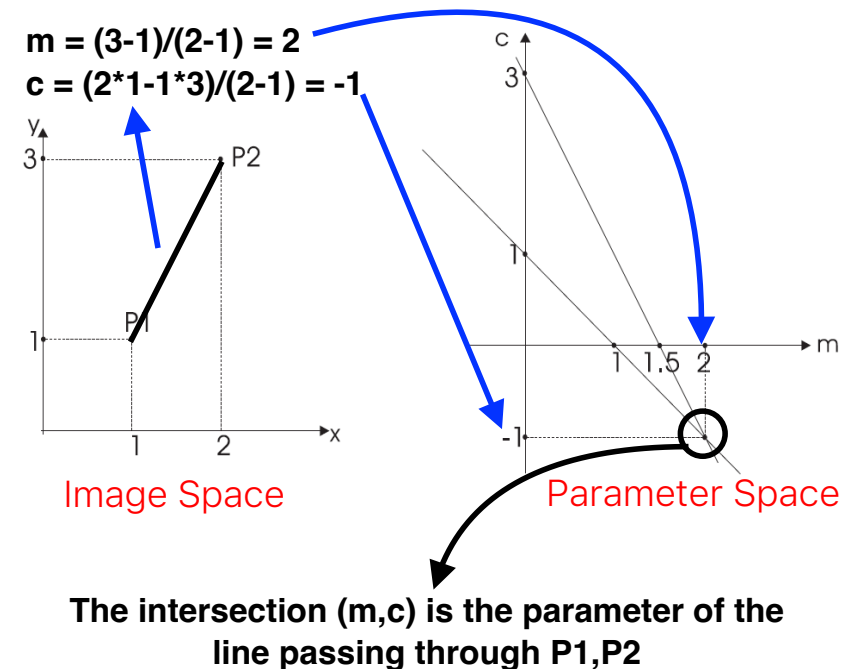
so the equation represents the mapping from image point $(\hat{x}, \hat{y})$ to the **parameter space** providing all the lines through the image point

Basic Principle

- Consider two image points P_1, P_2 and map both into the parameter space, we get two lines intersecting at the parameter space point representing the image line through P_1, P_2 :

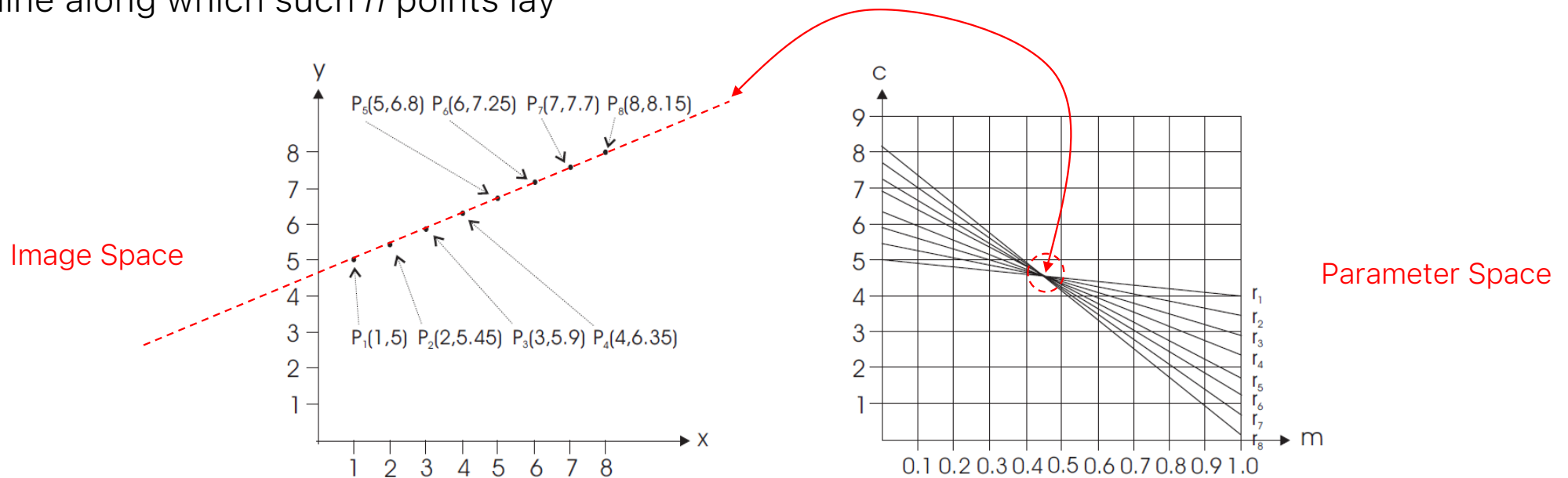
$$\begin{cases} \hat{y}_1 - m\hat{x}_1 - c = 0 \\ \hat{y}_2 - m\hat{x}_2 - c = 0 \end{cases} \Rightarrow \begin{cases} m = \frac{\hat{y}_2 - \hat{y}_1}{\hat{x}_2 - \hat{x}_1} \\ c = \frac{\hat{x}_2\hat{y}_1 - \hat{x}_1\hat{y}_2}{\hat{x}_2 - \hat{x}_1} \end{cases}$$

- m and c , represent the parameters of the line passing through P_1 and P_2
- More generally, if we map n image points we get as many intersections as $n(n-1)/2$ (i.e. the number of lines through the n image points)



Basic Principle

- Considering n collinear image points, we can notice that their corresponding transforms (i.e. parameter space lines) will intersect at a single parameter space point representing the image line along which such n points lay



- Rather than looking at an extended shape into the image, we look for a specific feature (where lines intersect) in the parameter space of lines

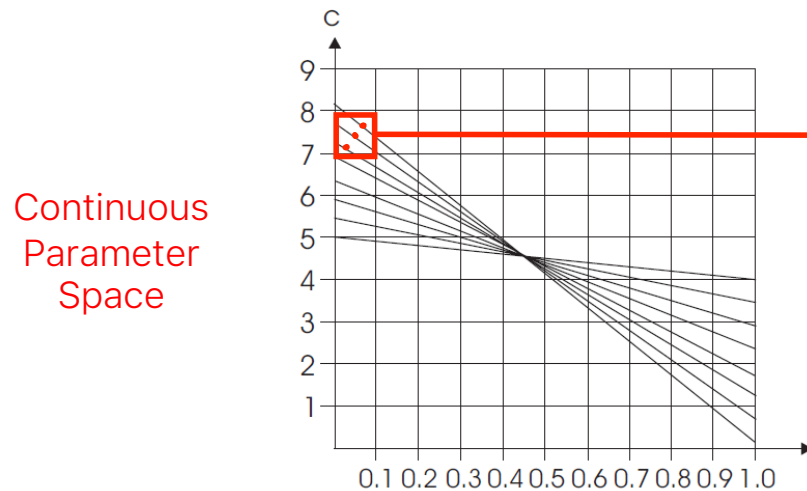
Basic Principle

- Therefore, given a sought *analytic shape represented by a set of parameters*, the HT consists in mapping image points (i.e. usually edge points) so as to create *curves into the parameter space of the shape*
- *Intersections of parameter space curves* indicate the presence of image points explained *by a certain instance of the shape*
 - the more the intersecting *curves* the more are such image points and thus the higher is the evidence of the presence of that instance in the image **Higher number of intersections ==> more important the instance of the curve**
- Detecting objects through the HT consists in finding *parameter space points through which many curves do intersect* (a local rather than global detection problem)
- To make it work in practice, the parameter space needs to be quantized and allocated as a memory array, which is often referred to as *Accumulator Array (AA)*
- Curves are “drawn” into the AA by a so called *voting process*:
 1. the transform equation is repeatedly computed to increment the bins satisfying the equation
 2. *a high number of intersecting curves at a point of the parameter space will provide a high number of votes* into a bin of the AA
 3. Finding parameter space points through which many curves do intersect is thus implemented in practice by finding *peaks of the AA*, i.e. local maxima showing a high number of votes

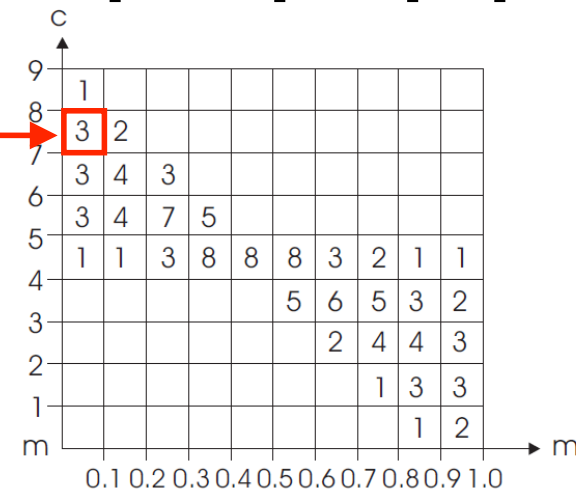
AA for line detection_(Voting)

We count the intersection as the number of lines in the quantized space (square)

- The AA highlights the presence of a line with



$$m \in [0.3, 0.6], c \in [4, 5]$$



- To detect the line more accurately, the AA should be quantized more finely
- The HT is robust to noise because spurious votes due to noise unlikely accumulate into a bin so as to trigger a false detection
- A partially occluded object can be detected provided that the threshold on the minimum number of votes required to declare a detection is lowered according to the degree of occlusion to be handled

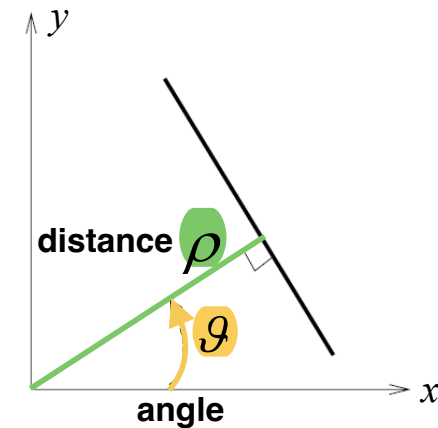
Partially occluded objects detected by: minimum threshold $\Leftarrow \Rightarrow$ degree of occlusion

HT for Line Detection

- The usual line parametrization considered so far (i.e. $y - mx - c = 0$) is impractical due to m spanning an infinite range
- The “normal parametrization” is adopted in the HT for lines: $\rho = x \cos \vartheta + y \sin \vartheta$

- Image points $(\hat{x}, \hat{y})$ are mapped into sinusoidal curves

of the (ϑ, ρ) parameter space: $\rho = \hat{x} \cos \vartheta + \hat{y} \sin \vartheta$



- With the normal parametrization: $\vartheta \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$, $\rho \in [-\rho_{\max}, \rho_{\max}]$

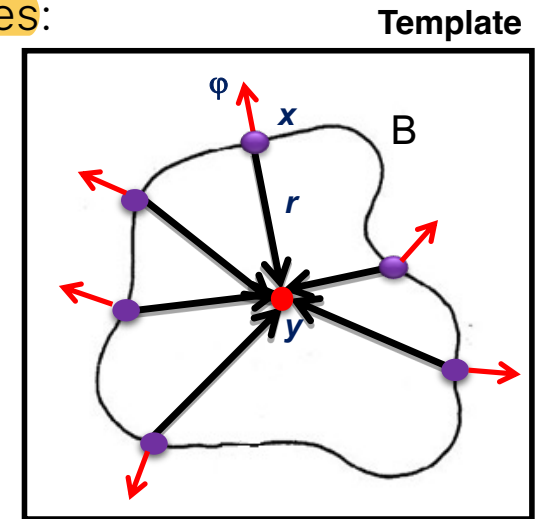
- while $\rho_{\max}$ is usually taken as large as the image diagonal: $N \times N$ pixels $\rightarrow \rho_{\max} = N \cdot \sqrt{2}$

Generalized Hough Transform

- The HT has been extended to detect arbitrary (i.e. non analytical) shapes:

- Off-line Phase (build the object's model) **Working on the template**

- A reference point y is chosen (e.g. barycentre)
- For each point x belonging to object's border B : **edge detection**
 - Compute gradient direction $\phi(x)$
 - Gradient direction is quantized according to a chosen step $\Delta\phi$
 - Compute vector r from y to x (i.e. $r = y - x$). **r = reference vector**
- Store r as a function of $\Delta\phi$ (R-Table)
- An entry in the R-Table can contain several r vectors



i	ϕ_i	R_{ϕ_i}
0	0	$\{r y - r = x, x \in B, \phi(x) = 0\}$
1	$\Delta\phi$	$\{r y - r = x, x \in B, \phi(x) = \Delta\phi\}$
2	$2\Delta\phi$	$\{r y - r = x, x \in B, \phi(x) = 2\Delta\phi\}$
...	...	...

We can obtain all the reference vectors within a given bin of directions

!! NOT ROTATION nor SCALE INVARIANT !!

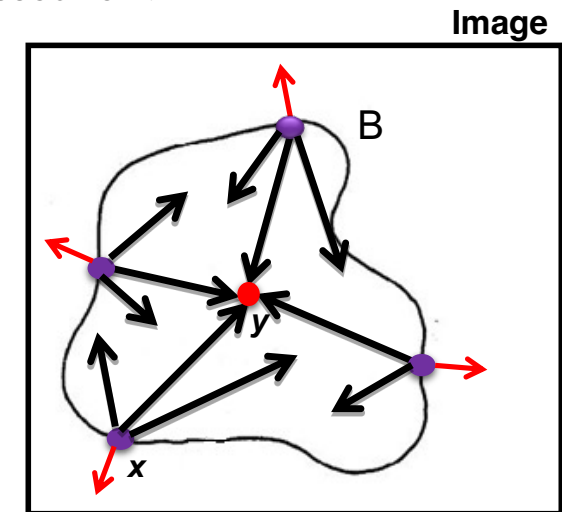
Overcome rotation variance by adding a dimension (which varies the rotation) to accumulator, even if in this way we would be looking for maxima in a volume.
The same also to cope with scale variance.

Generalized Hough Transform

- On-line Phase **Working on the image**

- We do edge detection first
- An image $A[y]$ is initialized as accumulator array. For each edge pixel x of the input image:
 - Compute gradient direction ϕ
 - Quantize ϕ to index the R-Table. For each r_i vector stored into the accessed row:
 - Compute the position of the reference point $y = x + r_i$
 - Cast a vote into the accumulator array $A[y]++$
- Instances of the sought object are detected by finding peaks of the accumulator array

i	ϕ_i	R_{ϕ_i}
0	0	$\{r y - r = x, x \in B, \phi(x) = 0\}$
1	$\Delta\phi$	r_1, r_2, r_3
2	$2\Delta\phi$	$\{r y - r = x, x \in B, \phi(x) = 2\Delta\phi\}$
...	...	...

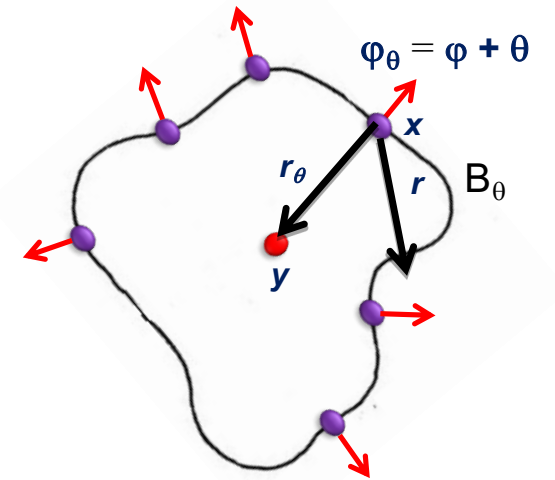
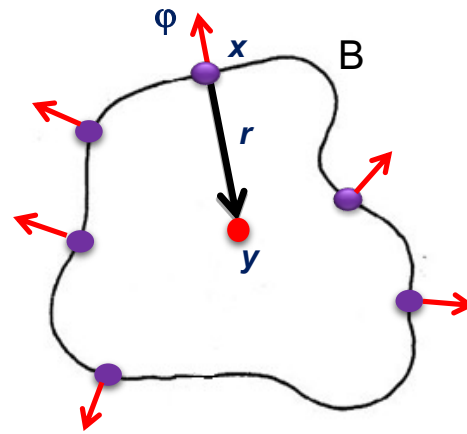


Where is the baricenter?

Given the x of the B and the corresponding reference vector, we compute where the baricenter y is projected and we give votes to the different baricenters that come out, in order to see if it is similar to the one of the template

Generalized Hough Transform

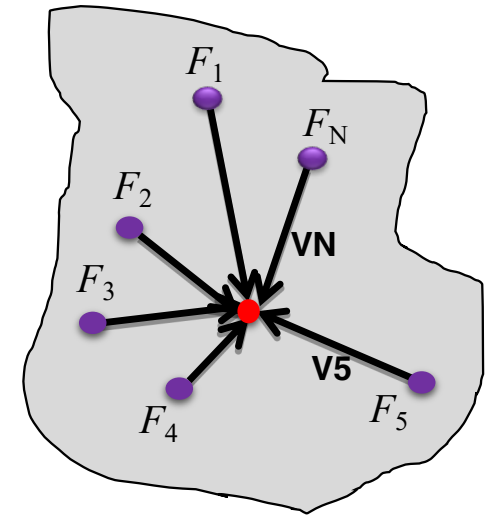
- Can we find the shape if it is rotated?



- We do not know θ , we should quantize rotation and try all of them!
- The same problem arises for scale

Geometric Validation: Star Model

- Base the GHT feature on local invariant features (e.g., SIFT) and not on edge detection
- **DoG features** are not found along contour (edge are pruned...) they **are found within the object**
- Off-line phase:
 - Detect features for each feature points
 - (position, canonical orientation, scale, descriptor)
 - Compute the baricenter and build a star model



$$F = \{F_1, F_2 \dots F_N\}, F_i = (\mathbf{P}_i, \varphi_i, S_i, \mathbf{D}_i) \quad \text{collect all the features } F_i$$

$$\text{Baricenter } \mathbf{P}_C = \frac{1}{N} \sum_{i=1}^N \mathbf{P}_i \rightarrow \mathbf{V}_i = \mathbf{P}_C - \mathbf{P}_i \quad \text{Joining vector: correspond to reference vectors of GHT}$$

- Add $\mathbf{V}_i \Rightarrow$ the joining vector $\forall F_i \in F$ $F_i = (\mathbf{P}_i, \varphi_i, S_i, \mathbf{D}_i, \mathbf{V}_i)$
- No longer use the R table

Geometric Validation

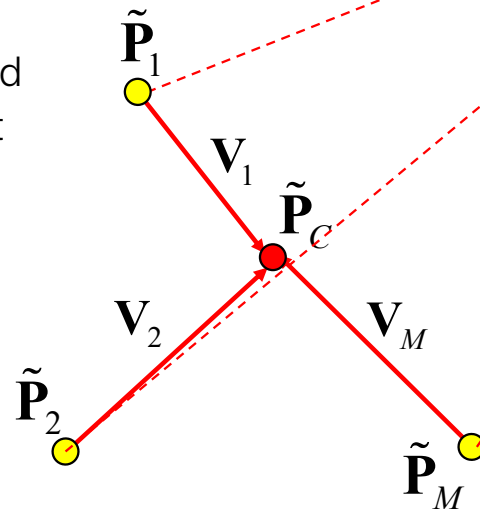
On-line phase

- Extract features from target image
 - Match features from the target image to the template
 - Now the matching is based on descriptor not gradient directions
- 

$$\tilde{F} = \left\{ \tilde{F}_1, \tilde{F}_2 \dots \tilde{F}_M \right\},$$

$$\tilde{F}_i = (\tilde{\mathbf{P}}_i, \tilde{\varphi}_i, \tilde{S}_i, \tilde{\mathbf{D}}_i)$$

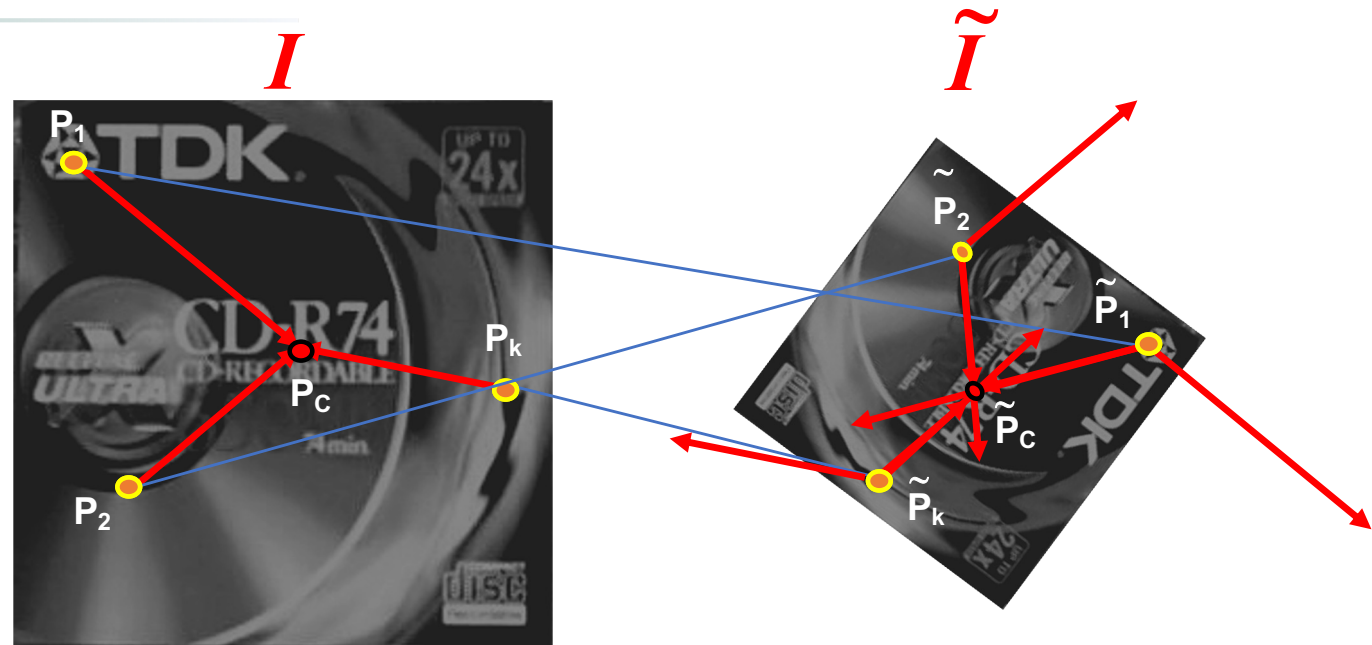
$$\tilde{D}_i \Leftrightarrow D_i, i = 1 \dots M$$



In theory we just look for the same P_i and using the respective V_i we rebuild the P_c and vote all the baricenters
But there is a problem with the orientations with a rotated template

Similarity Invariant Voting

- The joining vectors are not pointing coherently towards the reference point
- We should:
 - rotate all of them by the rotation obtained as the difference with the canonical rotation
 - Scale them according to the ratio of scales



$$F_i = (\mathbf{P}_i, \varphi_i, S_i, \mathbf{D}_i, \mathbf{V}_i)$$

$$\tilde{F}_i = (\tilde{\mathbf{P}}_i, \tilde{\varphi}_i, \tilde{S}_i, \tilde{\mathbf{D}}_i)$$

$$\Delta\varphi_i = \tilde{\varphi}_i - \varphi_i$$

$$s_i = \frac{\tilde{S}_i}{S_i}$$

$$\tilde{\mathbf{P}}_{C_i} = \tilde{\mathbf{P}}_i + s_i \cdot \mathbf{R}(\Delta\varphi_i) \mathbf{V}_i$$

Rotate $\mathbf{V}_i$
Scale $\mathbf{V}_i$

$$\mathbf{A}[\tilde{\mathbf{P}}_{C_i}] + +$$

Voting process

- Each match cast a vote for the position of the baricentre
- Use an accumulator array (quantized), increment each translation bin if a prediction hits a specific quantized position (voting process as before).
- The **accumulation array is 4 dimensional** because we need to accumulate also on the base of the rotation and scale hypothesis, otherwise we accumulate in the same translation bin but for different hypothesis of rotations and scales.

